

QAAgenticConfluence Home	2
SauceDemo eCommerce	3
1. Introduction & Project Overview	6
2. Business Context & Goals	10
3. System Overview & Architecture	13
4.1 Authentication & User Management	17
4.2 Product Catalog & Browsing	24
4.3 Shopping Cart	30
4.4 Checkout & Order Management	39
4.5 Admin / Inventory (future scope)	47
5. Non-Functional Requirements	52
5.1 Performance & Scalability	54
5.2 Security & Privacy	57
5.3 Reliability & Availability	62
5.4 Usability & Accessibility	65
5.5 Maintainability & Portability	68
5.6 Observability & Compliance	72
6. External Interfaces & Integrations	76
7. Acceptance Criteria & Test Strategy	79
8. Roadmap & Out-of-Scope (v2)	82
Requirements ↔ Jira Ticket Mapping	84

QAAgenticConfluence Home

Welcome to your new space!

Spaces help your team structure, organize, and share work, so every team member has visibility into institutional knowledge and access to the information they need to do their best work.

Get started with the basics

Start editing this content:

- ☐ Click the pencil icon  or **e** on your keyboard to edit and start typing. You can edit anywhere.
- ☐ Hit **/** to see all the types of content you can add to your page. Try **/image** or **/table**
- ☐ Use the toolbar at the top to play around with *font*, **colors**, ^{format}ing, and more
- ☐ Click **close** to save your draft or **publish** when your page is ready to be shared

Need some inspiration?

- Check out our [Confluence best practices guide](#).
- Get a quick intro into what spaces are and how to best use them at [Set up your site and spaces](#).
- Check out our guide for ideas on how to [set up your space overview](#).
- If starting from a blank space is daunting, try using one of the [space templates](#) instead.

SauceDemo eCommerce

SauceDemo eCommerce — Requirements Specification

Purpose of this space: Greenfield requirements wiki for the SauceDemo eCommerce application. Documents both **functional** and **non-functional** requirements as they would have been authored at project inception, prior to any code being written. Pages here serve as the source of truth for Jira epic / story creation, test design, and acceptance criteria.

About the application

SauceDemo eCommerce is a full-stack web application that lets shoppers authenticate, browse a product catalog, manage a shopping cart, and place orders. It comprises a Python **FastAPI** REST backend, a **React (Vite)** single-page frontend, and a relational database (SQLite in development, PostgreSQL in production).

Requirement ID conventions

Prefix	Meaning	Example
FR-<AREA>-NNN	Functional requirement	FR-CART-001
NFR-<AREA>-NNN	Non-functional requirement	NFR-SEC-003

Area codes:

Code	Area	Code	Area
AUTH	Authentication & User Management	PERF	Performance & Scalability
CAT	Product Catalog & Browsing	SEC	Security & Privacy
CART	Shopping Cart	REL	Reliability & Availability
ORD	Checkout & Order Management	USE	Usability & Accessibility
ADM	Admin / Inventory	MNT	Maintainability & Portability

		OBS	Observability & Compliance
--	--	-----	----------------------------

Each functional area maps to **one Jira Epic**; each numbered requirement maps to **one Story or Task** under that epic. Requirement IDs are stable independent of Jira project keys, so test cases and bug reports can reference them permanently.

Wiki structure

The requirements wiki is organized as a hierarchical tree of 13 pages under this landing page:

1. **1. Introduction & Project Overview** — purpose, scope, stakeholders, personas, glossary
2. **2. Business Context & Goals** — problem statement, success metrics, assumptions, constraints, MVP scope
3. **3. System Overview & Architecture** — high-level diagram, tech stack rationale, ERD
4. **4. Functional Requirements** (parent)
 - 4.1 Authentication & User Management — FR-AUTH-*
 - 4.2 Product Catalog & Browsing — FR-CAT-*
 - 4.3 Shopping Cart — FR-CART-*
 - 4.4 Checkout & Order Management — FR-ORD-*
 - 4.5 Admin / Inventory (future scope) — FR-ADM-*
5. **5. Non-Functional Requirements** (parent)
 - 5.1 Performance & Scalability — NFR-PERF-*
 - 5.2 Security & Privacy — NFR-SEC-*
 - 5.3 Reliability & Availability — NFR-REL-*
 - 5.4 Usability & Accessibility — NFR-USE-*
 - 5.5 Maintainability & Portability — NFR-MNT-*
 - 5.6 Observability & Compliance — NFR-OBS-*
6. **6. External Interfaces & Integrations** — REST API overview, planned third-party integrations
7. **7. Acceptance Criteria & Test Strategy** — golden paths, NFR test approach, test users
8. **8. Roadmap & Out-of-Scope (v2)**

Document status

Field	Value
-------	-------

Status	DRAFT
Version	0.1
Audience	Engineering, QA, Product, Stakeholders
Next step	Once approved, requirements drive Jira epic / story creation

How to read this wiki

- **Product / business readers** — start at pages 1, 2, then skim Functional Requirements summaries.
- **Engineers** — start at page 3 (System Overview), then dive into the functional area you own.
- **QA** — read all requirement pages, then page 7 (Test Strategy).
- **Reviewers** — use the requirement IDs in comments (e.g., "FR-CART-005 — should we allow quantity = 0 instead?").

1. Introduction & Project Overview

1. Introduction & Project Overview

Field	Value
Section	1 of 8
Audience	All stakeholders — read first
Status	DRAFT
Owner	Product Management

1.1 Purpose

This document specifies the functional and non-functional requirements for the **SauceDemo eCommerce** application. It captures the design intent of the system at project inception and serves as the source of truth for engineering work, QA test design, and stakeholder alignment.

The application is a web-based eCommerce platform that allows authenticated shoppers to browse products, manage a cart, and place orders. It is modeled on the publicly known SauceDemo demo store and is intended as a reference implementation suitable for QA training, automation practice, and small-scale production use.

1.2 Scope

In scope (MVP)

- User registration and JWT-based authentication
- Product catalog (browse, search, filter by category, view detail)
- Shopping cart (add, view, update, remove, clear; server-side persistence)
- Checkout and order creation
- Order history and order detail
- REST API with auto-generated OpenAPI documentation
- Responsive single-page application UI (mobile, tablet, desktop)

Out of scope (deferred to v2 — see Section 8)

- Payment gateway integration (Stripe, PayPal)
- Email and SMS notifications
- Admin dashboard for product / order management

- Product reviews, ratings, and wishlists
- OAuth / social login
- Multi-currency, multi-language
- Native mobile applications

1.3 Stakeholders

Role	Responsibility	Primary interest
Product Manager	Owns requirements; prioritises scope	Business outcomes, time to market
Engineering Lead	Architecture & delivery	Technical feasibility, debt
Frontend Engineers	React / Vite SPA	UI behaviour, API contract
Backend Engineers	FastAPI REST service	Endpoints, schema, persistence
QA Engineers	Test strategy, automation	Testability, acceptance criteria
DevOps / SRE	Deployment, observability	Reliability NFRs, infrastructure
Security Reviewer	Security NFRs	OWASP, PII, JWT hardening
End users (shoppers)	Use the application	UX, performance, reliability

1.4 User personas

Persona	Profile	Goals
Shopper Sarah (primary)	28, casual online shopper, mobile-first, varying network conditions	Quickly find products, compare, check out without friction
Returning Rick	Repeat customer who logs in across devices	Find previous orders, reuse shipping info, fast checkout
Admin Anita (future)	Catalog / order operations	Manage products, inventory, view all orders

QA Quinn	Test engineer using seeded test accounts	Reproducible flows, deterministic data, observable behaviour
-----------------	--	--

1.5 Glossary

Term	Definition
Cart	Per-user collection of pending purchase line items, persisted server-side
JWT	JSON Web Token — short-lived bearer credential issued at login
SPA	Single-Page Application — the React / Vite frontend
ORM	Object-Relational Mapper (SQLAlchemy)
MoSCoW	Prioritisation framework: Must / Should / Could / Won't
SHALL / SHOULD / MAY	Requirement strength per RFC 2119
Golden path	Happy-flow user journey used as the baseline test case
SLO	Service Level Objective — a measurable reliability target
p95 latency	95th-percentile response time
OWASP	Open Worldwide Application Security Project
WCAG	Web Content Accessibility Guidelines
ERD	Entity-Relationship Diagram

1.6 References

- RFC 2119 — Key words for requirement levels
- ISO/IEC 25010 — System and software quality models (NFR taxonomy)
- OWASP Top 10 (2021) — Web application security risks

- WCAG 2.1 — Web Content Accessibility Guidelines Level AA
- SauceDemo eCommerce landing page (parent of this document)

1.7 Document conventions

- Requirement IDs use the `FR-<AREA>-NNN` / `NFR-<AREA>-NNN` format (see landing page)
- Each requirement uses **SHALL / SHOULD / MAY** per RFC 2119
- Priority is annotated using MoSCoW: Must / Should / Could / Won't
- Code, paths, and identifiers are in `monospace`
- Mermaid diagrams are embedded as code blocks and render natively in this Confluence space

2. Business Context & Goals

2. Business Context & Goals

Field	Value
Section	2 of 8
Status	DRAFT
Owner	Product Management

2.1 Problem statement

Teams that learn QA automation, test strategy, and full-stack development need a realistic but compact eCommerce reference application — one that exhibits authentic flows (auth, catalog, cart, checkout) without the complexity of a production-grade retail platform. Existing public demos (such as the original SauceDemo) provide good test data but expose only the frontend; teams cannot exercise API contracts, backend authorisation, or end-to-end data flows on them.

SauceDemo eCommerce closes that gap by providing both a real REST API and a real SPA, deployable locally or to a small cloud footprint, with seeded test users and a well-documented data model.

2.2 Business objectives

1. Provide a reference implementation that exercises the full eCommerce request path end-to-end.
2. Enable QA teams to practice API, UI, integration, and load testing against a single application.
3. Demonstrate good software engineering practice (clear API contracts, typed schemas, separation of concerns).
4. Serve as the seed for future, more advanced eCommerce capabilities (payments, admin, recommendations).

2.3 Success metrics (KPIs)

Metric	Target	How measured
Successful test deployments	≥ 95% of attempts within 15 min of clone	Setup-guide walkthrough

End-to-end happy-path completion rate	$\geq 99\%$	Automated smoke test in CI
API p95 latency (auth, catalog, cart)	≤ 300 ms	Load test in QA env (see Section 7)
UI Lighthouse score	≥ 85 (Performance), ≥ 90 (Accessibility)	Lighthouse CI run
Test automation coverage	$\geq 80\%$ (backend), $\geq 70\%$ (frontend)	pytest / Vitest coverage reports
Critical security findings	0 (open)	SAST + manual review

2.4 Assumptions

- Users have modern evergreen browsers (Chrome, Firefox, Safari, Edge — last 2 major versions).
- Reliable network connectivity is the norm; degraded-network behaviour is in scope but offline-first is not.
- The relational database is the system of record for users, products, carts, and orders.
- Payment integration is deferred; v1 records orders without actual payment capture.
- The audience is internal (QA, engineering, training); v1 is not for general consumer use.

2.5 Constraints

Constraint	Detail
Technical	Python (FastAPI) backend, React/Vite frontend, SQLite (dev) / PostgreSQL (prod)
Operational	Must run locally with a single command for development
Security	JWT-only authentication for v1; no OAuth / social
Compliance	No real PII is processed; GDPR best practices observed where applicable
Budget	Open-source stack only; no paid SaaS dependencies in v1

Timeline	MVP releasable within a single training cohort cycle
----------	--

2.6 Risks

ID	Risk	Likelihood	Impact	Mitigation
R-01	JWT secret leakage	Low	High	Env-var-only secret; rotation policy (NFR-SEC-008)
R-02	SQLite limits hit under load	Medium	Medium	PostgreSQL upgrade path documented (Section 3)
R-03	Cart / inventory race conditions	Medium	Medium	Atomic decrement at checkout (FR-ORD-004)
R-04	Frontend bundle bloat	Medium	Low	Vite bundle budget, code-splitting (NFR-PERF-005)
R-05	Lack of monitoring in production	High	High	Structured logging from day 1 (NFR-OBS-001)

2.7 MVP definition

The MVP is releasable when:

- ☐ All **Must**-priority FRs in Section 4 are implemented and tested.
- ☐ All **Must**-priority NFRs in Section 5 meet their targets.
- ☐ All golden-path scenarios in Section 7 pass in CI.
- ☐ Security review completes with no open critical findings.
- ☐ Setup guide reproduces a working local environment for a new engineer.

3. System Overview & Architecture

3. System Overview & Architecture

Field	Value
Section	3 of 8
Audience	Engineering — start here
Status	DRAFT

3.1 High-level architecture

SauceDemo eCommerce is a classic three-tier web application: a browser-rendered React SPA communicates with a stateless FastAPI REST service that persists data in a relational database. Authentication is JWT-based; the SPA holds the token client-side and presents it on every request.

Error loading the extension!

Mermaid

```
1 flowchart LR
2   User((Shopper))
3   Browser[Browser<br>React SPA<br>Vite build]
4   API[FastAPI<br>REST service<br>Uvicorn]
5   DB[(Database<br>SQLite dev / PostgreSQL prod)]
6   Future[Stripe / Email<br>future v2]
7
8   User -->|HTTPS| Browser
9   Browser -->|REST + JWT| API
10  API --> DB
11  API -.future.-> Future
12  style Future stroke-dasharray: 5 5
```

3.2 Component responsibilities

Component	Responsibility	Tech
SPA (browser)	UI rendering, client routing, state, JWT storage, API calls	React 18, React Router, Axios, Context API
API service	Business logic, validation, persistence, JWT issuance & verification	FastAPI, Pydantic v2, SQLAlchemy ORM, python-jose, passlib/bcrypt

Database	System of record for users, products, carts, orders	SQLite (dev), PostgreSQL (prod)
API docs	Auto-generated interactive documentation	FastAPI Swagger UI (/docs), ReDoc (/redoc)

3.3 Tech stack & rationale

Layer	Choice	Why
Backend framework	FastAPI	Async-ready, Pydantic-validated, auto OpenAPI, low boilerplate
Backend language	Python 3.8+	Wide hiring pool; mature ecosystem
ORM	SQLAlchemy 2.x	Mature, dialect-portable (SQLite → Postgres)
Auth	JWT (HS256) via python-jose	Stateless, simple for SPA, no session store needed
Password hashing	bcrypt via passlib	Industry standard; slow by design
Frontend	React 18 + Vite 5	Modern hooks; instant HMR; small prod bundle
HTTP client	Axios + interceptor	Centralised JWT attach & 401 handling
Database (dev)	SQLite	Zero-setup for new contributors
Database (prod)	PostgreSQL	Concurrency, JSON columns, mature ops tooling

3.4 Data model (ERD)

Error loading the extension!

Mermaid

```
erDiagram
```

```

2  USER ||--o{ CART_ITEM : owns
3  USER ||--o{ ORDER : places
4  ORDER ||--|{ ORDER_ITEM : contains
5  PRODUCT ||--o{ CART_ITEM : referenced_by
6  PRODUCT ||--o{ ORDER_ITEM : referenced_by
7
8  USER {
9      int id PK
10     string username UK
11     string email UK
12     string hashed_password
13     string first_name
14     string last_name
15     bool is_active
16     datetime created_at
17 }
18 PRODUCT {
19     int id PK
20     string name
21     string description
22     decimal price
23     string image_url
24     string category
25     int inventory_count
26     bool is_active
27     datetime created_at
28 }
29 CART_ITEM {
30     int id PK
31     int user_id FK
32     int product_id FK
33     int quantity
34     datetime added_at
35 }
36 ORDER {
37     int id PK
38     int user_id FK
39     decimal total_amount
40     string status
41     string shipping_address
42     string payment_method
43     datetime created_at
44     datetime updated_at
45 }
46 ORDER_ITEM {
47     int id PK
48     int order_id FK
49     int product_id FK
50     int quantity
51     decimal price_at_purchase
52 }

```

3.5 Deployment topology (target)

Environment	Frontend	API	DB
Local dev	Vite dev server (5173)	Uvicorn (8000)	SQLite file
Staging	Static host (Vercel / Netlify)	Uvicorn + Gunicorn behind reverse proxy	Managed PostgreSQL

Production	CDN (Cloudflare / CloudFront)	Containerised API behind load balancer	Managed PostgreSQL with read replica
------------	-------------------------------	--	--------------------------------------

3.6 Cross-cutting concerns

- **Authentication** — all non-public endpoints require a valid JWT; verification is centralised in a FastAPI dependency.
- **Error handling** — standardised error envelope with HTTP status, code, and human-readable message.
- **Validation** — Pydantic models for all request and response bodies; rejection at API boundary, not in handlers.
- **CORS** — explicit allow-list per environment (local: localhost:5173).
- **Logging** — structured logs in JSON in production (see NFR-OBS-001).
- **Configuration** — environment variables only; no secrets in repo.

3.7 Future architecture considerations

- Async database sessions to remove blocking I/O on hot paths.
- Caching layer (Redis) for product catalog reads.
- Background task queue for email and order-fulfilment events.
- Service decomposition only if scaling forces it — start as a monolith.

4.1 Authentication & User Management

4.1 Authentication & User Management

Field	Value
Area code	AUTH
Jira Epic (planned)	ECOM-AUTH — Authentication & User Management
Status	DRAFT
Priority	PO / MVP
Dependencies	None (foundational)

1. Overview

The Authentication & User Management subsystem provides user registration, login, session continuity via JWT, and identity retrieval. It is foundational: every other subsystem (Cart, Orders) relies on the authenticated user identity it produces.

2. User stories

ID	As a...	I want...	So that...
US-AUTH-01	new visitor	register an account with email and password	I can start shopping and persist a cart
US-AUTH-02	returning user	log in with my username and password	I can resume where I left off
US-AUTH-03	logged-in user	see my own profile information	I can verify the right account is in use
US-AUTH-04	logged-in user	log out	my session is ended on shared devices
US-AUTH-05	system	reject expired or invalid tokens	protected endpoints stay protected

US-AUTH-06	future user	reset my password by email	I can recover access without contacting support
------------	-------------	----------------------------	---

3. Functional requirements

ID	Requirement	Story	Priority
FR-AUTH-001	The system SHALL allow a visitor to register a new account by providing username, email, password, first name, and last name.	US-AUTH-01	Must
FR-AUTH-002	The system SHALL enforce uniqueness of <code>username</code> and <code>email</code> , returning HTTP 409 on conflict.	US-AUTH-01	Must
FR-AUTH-003	The system SHALL hash passwords with bcrypt (≥ 12 rounds) before persistence. Plaintext passwords MUST NOT be logged or returned in any API response.	US-AUTH-01	Must
FR-AUTH-004	The system SHALL allow registered users to authenticate by submitting username and password, issuing a JWT on success.	US-AUTH-02	Must
FR-AUTH-005	The system SHALL return HTTP 401 for	US-AUTH-02	Must

	invalid credentials without revealing which of username / password was wrong.		
FR-AUTH-006	The system SHALL sign JWTs with HS256 using a server-side secret and SHALL set an expiry no longer than 30 minutes by default.	—	Must
FR-AUTH-007	The system SHALL provide an endpoint that returns the authenticated user's profile (id, username, email, first/last name).	US-AUTH-03	Must
FR-AUTH-008	The system SHALL reject any request that omits or presents an invalid / expired JWT on a protected endpoint with HTTP 401.	US-AUTH-05	Must
FR-AUTH-009	The frontend SHALL store the JWT in browser <code>localStorage</code> for v1 and SHALL clear it on logout.	US-AUTH-04	Must
FR-AUTH-010	The frontend SHALL automatically redirect users to the login page when an API response returns 401.	US-AUTH-05	Must

FR-AUTH-011	The system SHALL enforce password complexity: minimum 8 characters, at least one letter and one digit. Stronger rules MAY be added later.	US-AUTH-01	Must
FR-AUTH-012	The system SHALL provide three pre-seeded test users (<code>standard_user</code> , <code>problem_user</code> , <code>performance_user</code>) for automated testing.	—	Must
FR-AUTH-013	The system SHOULD provide a refresh-token mechanism to extend sessions without long-lived access tokens.	—	Should (v1.1)
FR-AUTH-014	The system SHOULD provide a password-reset flow via email link.	US-AUTH-06	Should (v1.1)
FR-AUTH-015	The system MAY support OAuth 2.0 social login (Google, GitHub) in a future release.	—	Could (v2)

4. Login sequence diagram

Error loading the extension!

▼ Mermaid

```
1 sequenceDiagram
```

```

2  actor User
3  participant SPA as React SPA
4  participant API as FastAPI
5  participant DB as Database
6
7  User->>SPA: Enter username + password
8  SPA->>API: POST /api/auth/login
9  API->>DB: SELECT user by username
10 DB-->>API: user row (with hashed_password)
11 API->>API: bcrypt.verify(password, hash)
12 alt valid credentials
13   API->>API: jwt.encode (sub user_id, secret, exp 30m)
14   API-->>SPA: 200 access_token bearer
15   SPA->>SPA: localStorage.setItem(token)
16   SPA->>API: GET /api/auth/me (Authorization Bearer)
17   API-->>SPA: 200 user profile
18   SPA-->>User: Redirect to products
19 else invalid credentials
20   API-->>SPA: 401 (no detail leaked)
21   SPA-->>User: Show generic error
22 end

```

5. API endpoints

Method	Path	Auth	Purpose	Maps to
POST	/api/auth/register	None	Create new user account	FR-AUTH-001/002/003/011
POST	/api/auth/login	None	Issue JWT for valid credentials	FR-AUTH-004/005/006
GET	/api/auth/me	JWT	Return authenticated user profile	FR-AUTH-007
POST	/api/auth/refresh	refresh-JWT	(v1.1) Issue new access token	FR-AUTH-013
POST	/api/auth/password-reset	None	(v1.1) Initiate password reset	FR-AUTH-014

6. UI / UX notes

- Login page displays test-user hints in non-production environments only.
- Registration form validates client-side for email format and password rules; server is authoritative.
- Generic "invalid username or password" message on failed login (FR-AUTH-005).
- Logout clears `localStorage`, clears in-memory auth context, redirects to the login page.

- An interceptor on the Axios client attaches the JWT to every request and watches for 401 responses (FR-AUTH-010).

7. Edge cases & error handling

Scenario	Expected behaviour	Requirement
Register with existing username or email	HTTP 409, generic message	FR-AUTH-002
Register with weak password	HTTP 422 with validation detail	FR-AUTH-011
Login with wrong password (correct user)	HTTP 401, generic message	FR-AUTH-005
Token expires mid-request	HTTP 401; SPA redirects to login	FR-AUTH-006/008/010
Token tampered (invalid signature)	HTTP 401	FR-AUTH-008
User account deactivated (<code>is_active = false</code>)	Login returns HTTP 401 (treat as invalid)	FR-AUTH-005

8. Acceptance criteria

- ☐ Registration creates a user with a bcrypt-hashed password (no plaintext anywhere).
- ☐ Duplicate username or email registration returns HTTP 409.
- ☐ Successful login returns a JWT whose `exp` is within 30 min.
- ☐ Invalid credentials return HTTP 401 with no leak about which field was wrong.
- ☐ `/api/auth/me` requires a JWT and returns the matching profile.
- ☐ Three seeded test users authenticate successfully.
- ☐ Frontend redirects to `/login` on any API 401.
- ☐ Password rule (8+, letter + digit) is enforced server-side.

9. Requirement ↔ Jira ticket mapping

To be populated when Jira tickets are created.

Requirement	Jira Epic	Jira Story / Task	Status
FR-AUTH-001 – FR-AUTH-015	ECOM-AUTH	TBD (one per requirement)	BACKLOG

10. Open questions

- Should we move JWT storage from `localStorage` to httpOnly cookies for v1.1? (Trade-off: CSRF vs XSS exposure.)
- Should we soft-lock accounts after N failed logins? (Pair with NFR-SEC rate limiting.)
- Should email be verified at registration (link in email) before login is permitted?

11. Related pages

- 3. System Overview & Architecture — JWT and User entity
- 4.3 Shopping Cart — consumer of authenticated identity
- 4.4 Checkout & Order Management — consumer of authenticated identity
- 5.2 Security & Privacy — JWT / password / rate-limiting NFRs

4.2 Product Catalog & Browsing

4.2 Product Catalog & Browsing

Field	Value
Area code	CAT
Jira Epic (planned)	ECOM-CAT
Status	DRAFT
Priority	PO / MVP
Dependencies	4.1 Authentication (for purchase actions only — browsing may be public)

1. Overview

The Product Catalog subsystem exposes the catalog of products that shoppers can browse, search, filter, and inspect in detail. It is read-mostly for shoppers; product creation / updates are admin-only (see 4.5).

2. User stories

ID	As a...	I want...	So that...
US-CAT-01	shopper	browse all active products in a grid	I can quickly survey what's available
US-CAT-02	shopper	search products by keyword	I can find a specific item
US-CAT-03	shopper	filter products by category	I can narrow to my area of interest
US-CAT-04	shopper	view the full details of a single product	I can make an informed buying decision
US-CAT-05	shopper	see whether a product is in stock	I don't add unavailable items

3. Functional requirements

ID	Requirement	Story	Priority
FR-CAT-001	The system SHALL list all active products (<code>is_active = true</code>) via a list endpoint.	US-CAT-01	Must
FR-CAT-002	The system SHALL support search over product <code>name</code> and <code>description</code> via a <code>search</code> query parameter (case-insensitive substring match in MVP).	US-CAT-02	Must
FR-CAT-003	The system SHALL support filtering products by <code>category</code> via a query parameter.	US-CAT-03	Must
FR-CAT-004	The system SHALL allow filters to combine (category AND search keyword).	US-CAT-02/03	Must
FR-CAT-005	The system SHALL provide a single-product retrieve endpoint returning all product attributes for a given <code>id</code> .	US-CAT-04	Must
FR-CAT-006	The system SHALL include each product's current <code>inventory_count</code>	US-CAT-05	Must

	(or a derived "in stock" boolean) in list and detail responses.		
FR-CAT-007	The system SHALL exclude inactive (<code>is_active = false</code>) products from list and search results.	—	Must
FR-CAT-008	Product list and detail endpoints SHALL be readable without authentication; mutating endpoints (create / update — see 4.5) SHALL require admin role.	—	Must
FR-CAT-009	The system SHALL ship with at least 6 seeded products spanning ≥ 2 categories, for testing and demos.	—	Must
FR-CAT-010	The system SHOULD paginate the product list once the catalog exceeds 50 products (cursor- or page-based).	—	Should (v1.1)
FR-CAT-011	The system SHOULD sort the product list by a deterministic default (e.g., <code>id</code> ascending) and SHOULD accept a <code>sort</code> query	—	Should

	parameter (price asc/desc, name).		
FR-CAT-012	The frontend SHALL display a placeholder image when <code>image_url</code> fails to load.	—	Should
FR-CAT-013	The system MAY return a "category facets" aggregation alongside list responses to power a counted filter UI.	—	Could

4. Catalog query flow

Error loading the extension!

▼ Mermaid

```

1 flowchart LR
2   UI[Catalog UI]
3   Q{search and/or category param?}
4   LIST[GET /api/products]
5   FILTER[Apply search and category filters server-side]
6   SORT[Sort by id ascending default]
7   OUT[Return JSON list]
8
9   UI --> Q
10  Q -- with params --> FILTER
11  Q -- no params --> LIST
12  LIST --> SORT
13  FILTER --> SORT
14  SORT --> OUT
15  OUT --> UI

```

5. API endpoints

Method	Path	Auth	Purpose	Maps to
GET	<code>/api/products</code>	None	List active products (filterable / searchable)	FR-CAT-001/002/003/004/007/010/011

GET	/api/products /{id}	None	Retrieve product detail	FR-CAT-005/006
-----	------------------------	------	-------------------------	----------------

Query parameters

Param	Type	Notes
search	string	Case-insensitive substring on name + description
category	string	Exact match
sort	enum	v1.1: price_asc / price_desc / name
page / limit	int	v1.1 pagination

6. UI / UX notes

- Catalog page presents a responsive grid (1 col mobile / 2–3 col tablet / 4 col desktop).
- Each tile shows image, name, price, category badge, in-stock indicator, "Add to cart" CTA.
- Search input debounces at ~300 ms and triggers a re-fetch.
- Category filter is a button row; selecting a category re-fetches.
- Product detail page shows full description, larger image, quantity selector, and "Add to cart".

7. Edge cases & error handling

Scenario	Expected behaviour	Requirement
Search returns zero results	UI shows empty-state with reset-filter CTA; API returns 200 with []	FR-CAT-002
Unknown category in query	API returns 200 with [] (no filter match)	FR-CAT-003
Product is deactivated between catalog load and detail click	Detail returns 404 (treat as not found)	FR-CAT-007
Image URL broken	UI shows placeholder	FR-CAT-012

Product detail invoked with unknown id	HTTP 404	FR-CAT-005
--	----------	------------

8. Acceptance criteria

- ☐ GET /api/products returns all active products (no inactive ones).
- ☐ ?search=keyword returns only matching products, case-insensitive.
- ☐ ?category=X returns only that category.
- ☐ Combined filters apply both conditions.
- ☐ GET /api/products/{id} returns 404 for unknown ids.
- ☐ Catalog shows ≥ 6 seeded products in ≥ 2 categories.
- ☐ Empty-state renders when no products match a filter.

9. Requirement ↔ Jira ticket mapping

Requirement	Jira Epic	Jira Story / Task	Status
FR-CAT-001 – FR-CAT-013	ECOM-CAT	TBD (one per requirement)	BACKLOG

10. Open questions

- Do we want fuzzy / tolerant search (Levenshtein) or strict substring in v1.1?
- Should product categories be a free-text string or a constrained enum?
- Do we surface inventory_count exactly, or only "in stock" / "low stock" / "out of stock"?

11. Related pages

- 4.3 Shopping Cart — consumer of product ids
- 4.5 Admin / Inventory — owner of product CRUD (v2)
- 3. System Overview & Architecture — Product entity in ERD

4.3 Shopping Cart

4.3 Shopping Cart

Field	Value
Area code	CART
Jira Epic (planned)	ECOM-CART — Shopping Cart
Status	DRAFT
Priority	PO / MVP
Dependencies	4.1 Authentication, 4.2 Product Catalog

1. Overview

The Shopping Cart subsystem lets an authenticated shopper collect products they intend to buy before checkout. Cart contents are persisted server-side, scoped per user, and survive across sessions and devices. The cart is the entry point to checkout (see *4.4 Checkout & Order Management*) and is automatically cleared after a successful order is placed.

i Scope of this page: add / view / update / remove cart items and clear cart. Pricing and tax computation occur at checkout time and are specified in 4.4.

2. User stories

ID	As a...	I want...	So that...
US-CART-01	shopper	add a product to my cart	I can buy multiple items in one transaction
US-CART-02	shopper	see my cart contents at any time	I know what I am about to purchase
US-CART-03	shopper	change the quantity of a cart item	I can buy more or fewer of a product without re-adding

US-CART-04	shopper	remove a single item from my cart	I do not accidentally buy items I changed my mind about
US-CART-05	shopper	clear my entire cart in one action	I can start over quickly
US-CART-06	shopper	my cart to persist when I log out and back in	I do not lose my selections
US-CART-07	shopper	see a live item-count badge in the header	I always know how many items are in my cart

3. Functional requirements

ID	Requirement	Story	Priority
FR-CART-001	The system SHALL allow an authenticated user to add a product to their cart by product ID and quantity.	US-CART-01	Must
FR-CART-002	When a user adds a product that already exists in their cart, the system SHALL increment the existing line's quantity rather than creating a duplicate line.	US-CART-01	Must
FR-CART-003	The system SHALL persist cart contents server-side, keyed to the authenticated user, so the cart	US-CART-06	Must

	survives across sessions and devices.		
FR-CART-004	The system SHALL allow a user to retrieve their current cart, returning per-line product details (name, image, unit price), line total, item count, and grand total.	US-CART-02	Must
FR-CART-005	The system SHALL allow a user to update the quantity of any cart line to any positive integer ≥ 1 .	US-CART-03	Must
FR-CART-006	The system SHALL reject quantity updates of 0 or negative values with HTTP 400; removal must use the delete endpoint.	US-CART-03	Must
FR-CART-007	The system SHALL allow a user to remove a single cart line.	US-CART-04	Must
FR-CART-008	The system SHALL allow a user to clear their entire cart in a single operation.	US-CART-05	Must
FR-CART-009	The system SHALL prevent adding a product whose <code>inventory_count</code> is 0 or whose	US-CART-01	Must

	<code>is_active</code> flag is false, returning HTTP 409 Conflict with a human-readable reason.		
FR-CART-010	The system SHALL display a real-time cart item-count badge in the application header on every authenticated page.	US-CART-07	Must
FR-CART-011	The system SHALL automatically clear the cart immediately after successful order creation (see FR-ORD-005).	—	Must
FR-CART-012	The system SHALL reject all cart endpoints when the request lacks a valid JWT, returning HTTP 401.	—	Must
FR-CART-013	The system SHALL cap any single cart line's quantity at the product's current <code>inventory_count</code> ; attempts to exceed this MUST return HTTP 409 with the available stock value.	US-CART-03	Should
FR-CART-014	The system SHOULD require an explicit confirmation prompt	US-CART-05	Should

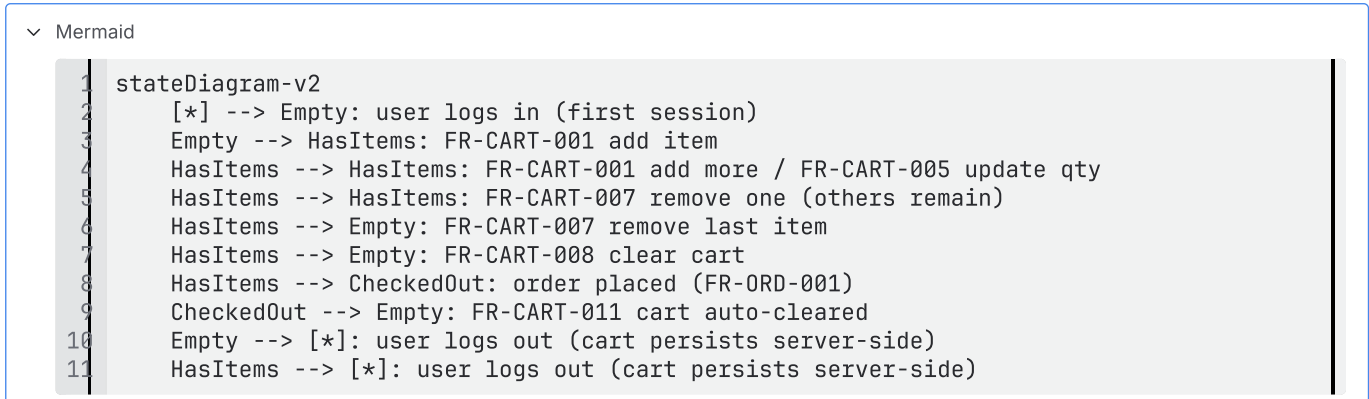
	on the client before invoking the clear-cart operation.		
FR-CART-015	The system MAY flag any cart line whose underlying product has since been deactivated or out-of-stock as "unavailable" and exclude it from the displayed grand total.	—	Could

Priority follows MoSCoW: **Must** = required for MVP, **Should** = important but deferrable, **Could** = nice-to-have.

4. Cart lifecycle (state diagram)

The cart for any given user transitions through the following states:

Error loading the extension!



📌 If Mermaid is not yet enabled in this Confluence space, the source above renders cleanly at mermaid.live. Once the Mermaid macro is installed, the diagram will render inline.

5. API endpoints

Method	Path	Auth	Purpose	Maps to
GET	/api/cart	JWT	Retrieve current user's cart with	FR-CART-004

			line items and totals	
POST	/api/cart/items	JWT	Add product to cart (or increment if already present)	FR-CART-001, 002, 009
PUT	/api/cart/items/{itemId}	JWT	Update line quantity	FR-CART-005, 006, 013
DELETE	/api/cart/items/{itemId}	JWT	Remove single line	FR-CART-007
DELETE	/api/cart	JWT	Clear entire cart	FR-CART-008

Request / response shapes (informative)

POST /api/cart/items

```

1 {
2   "product_id": 42,
3   "quantity": 1
4 }
```

GET /api/cart response

```

1 {
2   "items": [
3     {
4       "id": 101,
5       "product_id": 42,
6       "product_name": "Sauce Labs Backpack",
7       "image_url": "https://...",
8       "unit_price": 29.99,
9       "quantity": 2,
10      "line_total": 59.98,
11      "available": true
12    }
13  ],
14  "item_count": 2,
15  "grand_total": 59.98
16 }
```

6. UI / UX notes

- A cart icon with item-count badge is visible in the global header on every authenticated page (FR-CART-010). The badge updates optimistically on add / remove actions and is reconciled with the server response.
- Quantity controls on the cart page are `-` / `+` buttons surrounding an integer input, with the minimum displayed value being `1`. Clicking `-` at quantity `1` triggers item removal (not a 0-

quantity update).

- Removing a single item does **not** require confirmation — the cost of re-adding is low and accidental removals are easy to recover from.
- Clearing the whole cart **does** require a confirmation modal (FR-CART-014).
- Empty-cart state displays a friendly message ("Your cart is empty") and a CTA button linking to the product catalog.
- If a cart line is flagged "unavailable" (FR-CART-015), it is rendered with reduced opacity and a warning icon; the line is excluded from the grand total and from the checkout submission.

7. Edge cases & error handling

Scenario	Expected behaviour	Requirement
User adds a product that was deactivated since they viewed the catalog page	Return HTTP 409 with reason <code>PRODUCT_UNAVAILABLE</code> ; UI shows toast and refreshes catalog	FR-CART-009
User adds quantity exceeding remaining stock	Return HTTP 409 with payload <code>{available: N}</code> ; UI caps quantity at <code>N</code> and informs user	FR-CART-013
Product price changes between add-to-cart and checkout	Cart displays current price; final price-at-purchase is locked at order creation time (4.4)	FR-ORD-006
User edits cart on two devices concurrently	Last write wins; no merge logic in MVP. Client refetches cart on focus to reconcile.	FR-CART-003
Cart line references a product that has been hard-deleted	The line is omitted from the GET response and a server-side cleanup job removes the orphan record	FR-CART-015
Unauthenticated request to any cart endpoint	HTTP 401, no body content beyond standard error envelope	FR-CART-012

Authenticated but expired JWT	HTTP 401; client interceptor redirects to login page	FR-CART-012
-------------------------------	--	-------------

8. Acceptance criteria

- ☐ Adding the same product twice produces a single cart line with quantity = 2 (FR-CART-002).
- ☐ Cart contents survive a logout / login cycle on the same browser (FR-CART-003).
- ☐ Cart contents are visible after logging in on a different browser with the same user (FR-CART-003).
- ☐ Cart is empty immediately after a successful order is placed (FR-CART-011).
- ☐ All five cart endpoints return HTTP 401 when called without a token (FR-CART-012).
- ☐ Header cart badge reflects an add-to-cart action within 200 ms (FR-CART-010).
- ☐ Attempting to add a deactivated product returns HTTP 409 (FR-CART-009).
- ☐ PUT quantity = 0 returns HTTP 400 (FR-CART-006).
- ☐ Clear-cart action prompts for confirmation on the client (FR-CART-014).

9. Requirement ↔ Jira ticket mapping

To be populated when Jira tickets are created.

Requirement	Jira Epic	Jira Story / Task	Sprint	Status
FR-CART-001	ECOM-CART	TBD	—	BACKLOG
FR-CART-002	ECOM-CART	TBD	—	BACKLOG
FR-CART-003	ECOM-CART	TBD	—	BACKLOG
FR-CART-004	ECOM-CART	TBD	—	BACKLOG
FR-CART-005	ECOM-CART	TBD	—	BACKLOG
FR-CART-006	ECOM-CART	TBD	—	BACKLOG
FR-CART-007	ECOM-CART	TBD	—	BACKLOG
FR-CART-008	ECOM-CART	TBD	—	BACKLOG
FR-CART-009	ECOM-CART	TBD	—	BACKLOG
FR-CART-010	ECOM-CART	TBD	—	BACKLOG
FR-CART-011	ECOM-CART	TBD	—	BACKLOG
FR-CART-012	ECOM-CART	TBD	—	BACKLOG
FR-CART-013	ECOM-CART	TBD	—	BACKLOG

FR-CART-014	ECOM-CART	TBD	—	BACKLOG
FR-CART-015	ECOM-CART	TBD	—	BACKLOG

10. Open questions

- Should the cart auto-expire after N days of inactivity? (Suggested: 30 days; impacts FR-CART-003 .)
- Should we offer guest carts (cart before login)? Currently MVP requires auth — see FR-CART-012 .
- Should an "unavailable" cart line (FR-CART-015) be auto-removed after N days, or kept forever?

11. Related pages

- 4.1 Authentication & User Management — owners of the JWT enforced in FR-CART-012
- 4.2 Product Catalog & Browsing — source of `product_id` referenced in FR-CART-001
- 4.4 Checkout & Order Management — consumer of cart contents; trigger for FR-CART-011
- 3. System Overview & Architecture — data model showing `CartItem` entity

4.4 Checkout & Order Management

4.4 Checkout & Order Management

Field	Value
Area code	ORD
Jira Epic (planned)	ECOM-ORD
Status	DRAFT
Priority	PO / MVP
Dependencies	4.1 Authentication, 4.2 Catalog, 4.3 Cart

1. Overview

Checkout converts a cart into a persisted order and clears the cart. The order is the system of record for the purchase; it captures the line items, prices at the time of purchase, shipping address, and a placeholder payment method (real payment capture is deferred to v2 — see Section 6).

2. User stories

ID	As a...	I want...	So that...
US-ORD-01	shopper	review my cart, enter a shipping address, and place an order	I receive what I bought
US-ORD-02	shopper	see a confirmation of my order with a unique id	I know it succeeded and can reference it later
US-ORD-03	shopper	see a list of all my past orders	I can track my purchase history
US-ORD-04	shopper	open any past order to see line items and totals	I can verify or reorder

US-ORD-05	future shopper	cancel an order before it ships	I can change my mind
-----------	----------------	---------------------------------	----------------------

3. Functional requirements

ID	Requirement	Story	Priority
FR-ORD-001	The system SHALL allow an authenticated user with a non-empty cart to create an order by submitting shipping address and payment method.	US-ORD-01	Must
FR-ORD-002	The system SHALL reject order creation when the user's cart is empty, returning HTTP 400.	—	Must
FR-ORD-003	The system SHALL validate that every cart line refers to an active product with sufficient inventory at the time of order creation; insufficient stock returns HTTP 409.	—	Must
FR-ORD-004	The system SHALL atomically decrement <code>inventory_count</code> for each line as part of order creation; the operation MUST either fully succeed or fully roll back.	—	Must

FR-ORD-005	The system SHALL clear the user's cart on successful order creation (see FR-CART-011).	—	Must
FR-ORD-006	The system SHALL lock the unit price of each line into <code>OrderItem.price_at_purchase</code> at order creation time; subsequent product price changes MUST NOT alter historic orders.	—	Must
FR-ORD-007	The system SHALL compute and store <code>total_amount</code> as the sum of <code>price_at_purchase × quantity</code> over all order items.	—	Must
FR-ORD-008	The system SHALL list all orders belonging to the authenticated user, sorted by <code>created_at</code> descending.	US-ORD-03	Must
FR-ORD-009	The system SHALL provide order detail (line items, totals, address, status, timestamps) for any order owned by the authenticated user.	US-ORD-04	Must

FR-ORD-010	The system SHALL deny access to orders that do not belong to the authenticated user, returning HTTP 404 (not 403, to avoid leaking existence).	—	Must
FR-ORD-011	The system SHALL assign an initial order status of PENDING on creation and SHALL allow the lifecycle states: PENDING → CONFIRMED → SHIPPED → DELIVERED , plus CANCELLED as a terminal alternative.	—	Must
FR-ORD-012	The system SHALL persist the shipping address as captured at checkout, separately from the user's profile (a user may ship to many addresses).	—	Must
FR-ORD-013	The system SHOULD allow a user to cancel an order while its status is PENDING or CONFIRMED ; cancellation MUST return reserved inventory.	US-ORD-05	Should

FR-ORD-014	The system SHOULD support an idempotency key on order creation so retried submissions do not produce duplicate orders.	—	Should
FR-ORD-015	The system MAY send an email confirmation on successful order creation in v1.1.	—	Could

4. Checkout sequence diagram

Error loading the extension!

▼ Mermaid

```

1 sequenceDiagram
2     actor User
3     participant SPA as React SPA
4     participant API as FastAPI
5     participant DB as Database
6
7     User->>SPA: Click "Place order"
8     SPA->>API: POST /api/orders address payment_method
9     API->>DB: BEGIN tx
10    API->>DB: SELECT cart items for user
11    API->>DB: SELECT products FOR UPDATE
12    Note over API,DB: validate stock per line
13    alt stock OK
14        API->>DB: INSERT order
15        API->>DB: INSERT order_items snapshot prices
16        API->>DB: UPDATE products set inventory minus qty
17        API->>DB: DELETE cart_items WHERE user_id
18        API->>DB: COMMIT
19        API-->>SPA: 201 order_id total
20        SPA-->>User: Show confirmation page
21    else stock NOT OK
22        API->>DB: ROLLBACK
23        API-->>SPA: 409 conflict out_of_stock
24        SPA-->>User: Show error link to cart
25    end
end

```

5. Order status lifecycle

Error loading the extension!

▼ Mermaid

```

1 stateDiagram-v2
2     [*] --> PENDING: FR-ORD-001 order created
3     PENDING --> CONFIRMED: payment verified (v2)
4     PENDING --> CANCELLED: FR-ORD-013 user cancels
5     CONFIRMED --> SHIPPED: fulfilment event

```

```

6   CONFIRMED --> CANCELLED: FR-ORD-013 user cancels
7   SHIPPED --> DELIVERED: tracking confirms
8   DELIVERED --> [*]
9   CANCELLED --> [*]

```

6. API endpoints

Method	Path	Auth	Purpose	Maps to
POST	/api/orders	JWT	Create order from current cart	FR-ORD-001/002/003/004/005/006/007/011/012
GET	/api/orders	JWT	List orders for current user	FR-ORD-008/010
GET	/api/orders/{id}	JWT	Order detail	FR-ORD-009/010
POST	/api/orders/{id}/cancel	JWT	(v1.1) Cancel order, restore stock	FR-ORD-013

7. UI / UX notes

- Checkout page summarises the cart (read-only), accepts shipping address and a payment-method selector.
- The "Place order" button is disabled until required fields are valid.
- On success, the user lands on an order-confirmation page showing the order id and totals.
- The Orders page lists past orders with status badges; each row expands to show line items.

8. Edge cases & error handling

Scenario	Expected behaviour	Requirement
Cart empty at checkout	HTTP 400 with reason	FR-ORD-002
Product went out of stock between add-to-cart and checkout	HTTP 409; user is sent back to cart with the offending line flagged	FR-ORD-003

Two simultaneous checkouts on the same low-stock product	One succeeds, one fails atomically (no oversell)	FR-ORD-004
User requests an order id that belongs to someone else	HTTP 404 (do not leak existence)	FR-ORD-010
User retries POST /orders after a network blip	(v1.1) Idempotency key returns the original order id	FR-ORD-014
Cancel an order already shipped	HTTP 409 with reason	FR-ORD-013

9. Acceptance criteria

- ☐ POST /orders creates an order with status PENDING and snapshots prices.
- ☐ Cart is empty immediately after successful POST /orders.
- ☐ Inventory decremented exactly once per item per order.
- ☐ Concurrent checkouts cannot oversell the last unit.
- ☐ GET /orders returns only the caller's orders.
- ☐ GET /orders/{id} for another user's order returns 404.
- ☐ total_amount equals the sum of price_at_purchase × quantity.
- ☐ Empty-cart checkout returns 400.

10. Requirement ↔ Jira ticket mapping

Populated alongside Jira ticket creation.

Requirement	Jira Epic	Jira Story / Task	Status
FR-ORD-001 – FR-ORD-015	ECOM-ORD	TBD	BACKLOG

11. Open questions

- Should sales tax / shipping cost be computed at this stage, or deferred to v2 alongside Stripe?
- Should shipping address be selectable from a user-owned "address book" (separate entity) instead of free text?
- Should cancellation auto-trigger a refund event when payments arrive in v2?

12. Related pages

- 4.3 Shopping Cart — supplies the cart cleared by FR-ORD-005
- 4.1 Authentication & User Management — user identity
- 4.5 Admin / Inventory — admin view of all orders (v2)
- 6. External Interfaces & Integrations — Stripe sketch (v2)

4.5 Admin / Inventory (future scope)

4.5 Admin / Inventory (future scope)

⚠ **This area is v2 / future scope.** No admin UI exists in the MVP. The endpoints listed here document the intended shape so MVP work does not foreclose them.

Field	Value
Area code	ADM
Jira Epic (planned)	ECOM-ADM
Status	FUTURE / V2
Priority	V2
Dependencies	Role-based authorisation (extension of 4.1)

1. Overview

The Admin / Inventory subsystem gives operational staff control over the catalog, orders, and inventory. It introduces role-based access control: a user with role **ADMIN** can call mutating endpoints that shoppers cannot.

2. User stories

ID	As an...	I want...	So that...
US-ADM-01	admin	create, update, and deactivate products	I can curate the catalog
US-ADM-02	admin	adjust inventory counts	I can correct discrepancies after restocks or audits
US-ADM-03	admin	view all orders across all users	I can support fulfilment and customer service
US-ADM-04	admin	update order status (CONFIRMED /	I can reflect fulfilment progress

		SHIPPED / DELIVERED)	
US-ADM-05	admin	deactivate user accounts	I can handle abuse or account closure requests

3. Functional requirements

ID	Requirement	Story	Priority
FR-ADM-001	The system SHALL introduce a <code>role</code> attribute on User with values <code>USER</code> (default) and <code>ADMIN</code> .	—	Must (v2)
FR-ADM-002	The system SHALL allow ADMIN users to create products.	US-ADM-01	Must (v2)
FR-ADM-003	The system SHALL allow ADMIN users to update any field of an existing product.	US-ADM-01	Must (v2)
FR-ADM-004	The system SHALL allow ADMIN users to soft-delete (deactivate) a product by setting <code>is_active = false</code> ; hard deletion is NOT permitted to preserve historic order references.	US-ADM-01	Must (v2)
FR-ADM-005	The system SHALL allow ADMIN users to adjust	US-ADM-02	Must (v2)

	<code>inventory_count</code> directly, recording an audit-log entry with old/new values and timestamp.		
FR-ADM-006	The system SHALL allow ADMIN users to list all orders across all users, with pagination and filtering by status.	US-ADM-03	Must (v2)
FR-ADM-007	The system SHALL allow ADMIN users to transition order status (per FR-ORD-011 lifecycle).	US-ADM-04	Must (v2)
FR-ADM-008	The system SHALL reject all admin endpoints from non-ADMIN users with HTTP 403.	—	Must (v2)
FR-ADM-009	The system SHOULD allow ADMIN users to deactivate user accounts (set <code>is_active = false</code>); active sessions of deactivated users SHALL be invalidated at next request.	US-ADM-05	Should (v2)
FR-ADM-010	The system SHOULD provide admin-only metrics endpoints (total orders, revenue	—	Should (v2)

	last N days, top products).		
FR-ADM-011	The system MAY provide a CSV export of orders for a given date range.	—	Could (v2)
FR-ADM-012	All admin actions MUST be recorded in an audit log (who, when, what changed).	—	Must (v2)

4. API endpoints (planned)

Method	Path	Auth	Purpose	Maps to
POST	/api/products	JWT + ADMIN	Create product	FR-ADM-002
PUT	/api/products/{id}	JWT + ADMIN	Update product	FR-ADM-003
DELETE	/api/products/{id}	JWT + ADMIN	Soft-deactivate product	FR-ADM-004
PATCH	/api/products/{id}/inventory	JWT + ADMIN	Adjust inventory_count	FR-ADM-005
GET	/api/admin/orders	JWT + ADMIN	List all orders	FR-ADM-006
PATCH	/api/admin/orders/{id}/status	JWT + ADMIN	Transition order status	FR-ADM-007
PATCH	/api/admin/users/{id}/deactivate	JWT + ADMIN	Deactivate user	FR-ADM-009

GET	/api/admin/metrics	JWT + ADMIN	Operational metrics	FR-ADM-010
-----	--------------------	-------------	---------------------	------------

5. Acceptance criteria

- ☐ Non-admin user calling any admin endpoint receives HTTP 403.
- ☐ Admin can create, update, deactivate a product.
- ☐ Soft-deleted products remain referenced by historic order lines.
- ☐ Inventory adjustments are recorded in the audit log.
- ☐ Admin can list and update all orders.
- ☐ Deactivated user's subsequent requests return 401.

6. Requirement ↔ Jira ticket mapping

Requirement	Jira Epic	Jira Story / Task	Status
FR-ADM-001 – FR-ADM-012	ECOM-ADM	TBD	BACKLOG (V2)

7. Open questions

- Should the admin surface be a separate SPA, a separate route prefix in the same SPA, or a dedicated tool entirely?
- Do we need fine-grained roles (catalog-only, order-only, super-admin) or just a single ADMIN role for v2?
- Where does the audit log live — same DB or a separate append-only store?

8. Related pages

- 4.1 Authentication — extends with role attribute
- 4.2 Product Catalog — owns the read side of products
- 4.4 Checkout & Order Management — owns the order lifecycle (admin transitions a subset)
- 5.2 Security & Privacy — RBAC NFRs

5. Non-Functional Requirements

5. Non-Functional Requirements

 This page is the index of all non-functional requirements (NFRs) — also known as *quality attributes*. Each NFR sub-area has its own child page with measurable targets and verification approach.

5.0 Overview

Non-functional requirements describe *how well* the system performs — performance, security, reliability, usability, maintainability, and observability. Unlike functional requirements, NFRs are usually expressed as measurable thresholds (e.g., "p95 latency $\leq$ 300 ms under 100 concurrent users").

NFR categories below follow the **ISO/IEC 25010** quality model.

5.1 NFR category index

#	Quality attribute	ID prefix	Why it matters
5.1	Performance & Scalability	NFR-PERF-*	Responsive UX under expected and peak load
5.2	Security & Privacy	NFR-SEC-*	Protect credentials, data, and system integrity
5.3	Reliability & Availability	NFR-REL-*	Service is up when users need it; degrades gracefully
5.4	Usability & Accessibility	NFR-USE-*	Everyone can use the app, including assistive-tech users
5.5	Maintainability & Portability	NFR-MNT-*	Easy to evolve; runs across environments
5.6	Observability & Compliance	NFR-OBS-*	We can see what's happening; we meet

5.2 NFR target summary (headline numbers)

Attribute	Target	Detail page
API p95 latency	≤ 300 ms	5.1
Concurrent user capacity	≥ 200 in MVP	5.1
OWASP Top 10 coverage	100%	5.2
Password hashing	bcrypt, ≥ 12 rounds	5.2
Uptime SLO	99.5%	5.3
Accessibility	WCAG 2.1 AA	5.4
Test coverage	$\geq 80\%$ backend / 70% frontend	5.5
Structured logging	100% of API requests	5.6

5.3 NFR verification approach

- **Performance** — load tests with k6 or Locust against staging; published baseline.
- **Security** — automated SAST (Bandit, npm audit) + manual review checklist; penetration test before any external launch.
- **Reliability** — chaos drills (restart API mid-request); backup / restore exercise.
- **Usability** — Lighthouse Accessibility scan; manual screen-reader run; tab-only navigation pass.
- **Maintainability** — coverage gates in CI; lint and type checks block merge.
- **Observability** — log sampling review; alert-rule firing exercise.

5.1 Performance & Scalability

5.1 Performance & Scalability

Field	Value
Category code	PERF
ISO 25010 attribute	Performance efficiency
Status	DRAFT

1. Overview

This page captures performance and scalability targets, the load profile they assume, and the verification approach.

2. Load profile assumed for MVP

Metric	MVP target	Notes
Concurrent active users	200	Mixed read/write traffic
Catalog reads per user per session	20	Includes search / filter calls
Cart mutations per user per session	10	Add / update / remove
Checkout rate	1 per user per session	
Peak read RPS	~100	Catalog-heavy
Peak write RPS	~20	Cart + order

3. Non-functional requirements

ID	Requirement	Target	Priority
NFR-PERF-001	API endpoints SHALL respond within target latency under the MVP load profile.	p50 ≤ 100 ms, p95 ≤ 300 ms, p99 ≤ 800 ms	Must

NFR-PERF-002	Catalog read endpoints SHALL sustain the peak read RPS without queueing.	≥ 100 RPS sustained	Must
NFR-PERF-003	Checkout endpoint SHALL complete within target latency including its database transaction.	$p95 \leq 500$ ms	Must
NFR-PERF-004	The frontend SHALL achieve Lighthouse Performance ≥ 85 on a mid-tier mobile profile.	≥ 85	Must
NFR-PERF-005	The frontend production bundle (gzipped, initial route) SHALL be ≤ 200 KB.	≤ 200 KB	Must
NFR-PERF-006	First Contentful Paint SHALL be ≤ 1.5 s on a fast-3G simulated connection.	≤ 1.5 s	Should
NFR-PERF-007	The system SHOULD use response caching (Cache-Control headers) on the catalog list endpoint for at least 60 s.	≥ 60 s	Should
NFR-PERF-008	The API SHOULD scale horizontally: multiple stateless API instances behind a load balancer with the database as the only shared state.	≥ 4 instances tested	Should

NFR-PERF-009	The system SHOULD introduce a Redis cache for catalog reads in v1.1, with cache invalidation on admin writes.	—	Should (v1.1)
NFR-PERF-010	The catalog list endpoint SHALL paginate at > 50 products (see FR-CAT-010).	page size ≤ 50	Must (v1.1)

4. Verification approach

- Run scripted load profile (k6 or Locust) against staging; capture p50/p95/p99 per endpoint.
- Publish a baseline result. Subsequent runs gate PR merges if any percentile regresses > 10%.
- Frontend: run Lighthouse CI on every PR.
- Bundle size: enforce budget in CI (vite-bundle-visualizer or size-limit).

5. Open questions

- What is the long-term scalability target — 1k concurrent users? 10k? Determines DB sizing and caching strategy.
- Do we need CDN-fronted static assets for v1, or is direct host serving acceptable?

6. Requirement ↔ Jira ticket mapping

Requirement	Jira Epic	Status
NFR-PERF-001 – 010	ECOM-NFR-PERF	BACKLOG

7. Related pages

- 3. System Overview & Architecture — caching and scaling topology
- 5.3 Reliability & Availability — load shedding under stress
- 7. Acceptance Criteria & Test Strategy — load test tooling

5.2 Security & Privacy

5.2 Security & Privacy

Field	Value
Category code	SEC
ISO 25010 attribute	Security
Status	DRAFT

1. Overview

Security NFRs are derived from OWASP Top 10 (2021) and standard practice for JWT-based REST APIs. PII handling assumes GDPR-style defaults even when not strictly required.

2. Non-functional requirements

ID	Requirement	Detail	Priority
NFR-SEC-001	All traffic SHALL be served over HTTPS (TLS 1.2+) in any non-local environment.	Enforce HSTS	Must
NFR-SEC-002	Passwords SHALL be hashed with bcrypt at ≥ 12 rounds.	passlib config	Must
NFR-SEC-003	JWT secret SHALL be ≥ 256 -bit random entropy, sourced only from environment variables, never committed to source control.	—	Must
NFR-SEC-004	JWT access tokens SHALL expire in ≤ 30 minutes; refresh	—	Must

	tokens (v1.1) SHALL expire in ≤ 7 days.		
NFR-SEC-005	All user input SHALL be validated via Pydantic at the API boundary; SQL access SHALL go exclusively through SQLAlchemy ORM to prevent injection.	—	Must
NFR-SEC-006	CORS SHALL be configured with an explicit origin allow-list per environment; wildcard is forbidden.	—	Must
NFR-SEC-007	Error responses SHALL NOT include stack traces or internal identifiers in production.	—	Must
NFR-SEC-008	Secrets (JWT signing key, database password) SHALL be rotated at least every 90 days with documented procedure.	Runbook	Should
NFR-SEC-009	The system SHALL apply rate limiting on authentication endpoints (e.g., 5 attempts per minute per IP and per username).	—	Must (v1.1)

NFR-SEC-010	The system SHALL set security headers in API and frontend responses: X-Content-Type-Options, Strict-Transport-Security, Referrer-Policy, frame-deny.	—	Must
NFR-SEC-011	User PII (email, name, address) SHALL be exposed only via authenticated endpoints scoped to the owning user (see FR-ORD-010 ownership pattern).	—	Must
NFR-SEC-012	The system SHALL provide a documented data-deletion procedure on user request (GDPR Article 17).	Runbook	Should
NFR-SEC-013	Logs SHALL NOT contain passwords, JWTs, payment data, or full email addresses (mask before log).	—	Must
NFR-SEC-014	Dependencies SHALL be scanned for known CVEs in CI (pip-audit / npm audit). Critical findings SHALL block merge.	—	Must

NFR-SEC-015	Static analysis (Bandit, ESLint with security plugin) SHALL run in CI.	—	Must
-------------	--	---	------

3. OWASP Top 10 (2021) mapping

OWASP risk	Mitigated by
A01 Broken access control	FR-AUTH-008, FR-ORD-010, FR-ADM-008, NFR-SEC-011
A02 Cryptographic failures	NFR-SEC-001/002/003/004
A03 Injection	NFR-SEC-005
A04 Insecure design	Greenfield design via this wiki; reviewed
A05 Security misconfiguration	NFR-SEC-006/010, NFR-SEC-007
A06 Vulnerable & outdated components	NFR-SEC-014
A07 Identification & auth failures	FR-AUTH-005/006/011, NFR-SEC-002/004/009
A08 Software & data integrity failures	NFR-SEC-014/015
A09 Security logging & monitoring	NFR-SEC-013, NFR-OBS-005
A10 SSRF	No outbound user-controlled URLs in MVP

4. Verification approach

- SAST in CI (Bandit, ESLint security plugin).
- Dependency CVE scan on every build (pip-audit, npm audit).
- Manual review against this NFR list before any external launch.
- OWASP ZAP baseline scan against staging on a weekly cron.
- Threat-model session at design time (this wiki suffices for MVP).

5. Open questions

- Cookie vs localStorage for JWT — security posture decision (CSRF vs XSS) needs ratification.

- Do we encrypt PII at rest above DB-level (per-column encryption)?
- Do we require WAF in front of the API in production?

6. Requirement ↔ Jira ticket mapping

Requirement	Jira Epic	Status
NFR-SEC-001 – 015	ECOM-NFR-SEC	BACKLOG

7. Related pages

- 4.1 Authentication & User Management — concrete auth mechanism
- 5.6 Observability & Compliance — audit logging
- 7. Acceptance Criteria & Test Strategy — security test approach

5.3 Reliability & Availability

5.3 Reliability & Availability

Field	Value
Category code	REL
ISO 25010 attribute	Reliability
Status	DRAFT

1. Overview

Reliability NFRs target uptime, graceful degradation, recovery from failure, and data durability.

2. Non-functional requirements

ID	Requirement	Target	Priority
NFR-REL-001	The system SHALL maintain $\geq 99.5\%$ monthly uptime in production (max ~ 3.6 h downtime / month).	$\geq 99.5\%$	Must
NFR-REL-002	The API SHALL expose a liveness endpoint (/health/live) and a readiness endpoint (/health/ready) for load-balancer and orchestrator integration.	—	Must
NFR-REL-003	Order creation SHALL be transactional: stock decrement, order insert, and cart clear	—	Must

	MUST all commit or all roll back (see FR-ORD-004).		
NFR-REL-004	The database SHALL be backed up at least daily with point-in-time recovery enabled; backups SHALL be tested by restore drill at least quarterly.	RPO $\leq$ 24 h, restore drill quarterly	Must
NFR-REL-005	Mean Time To Recover (MTTR) from a complete service outage SHALL be $\leq$ 30 minutes.	$\leq$ 30 min	Must
NFR-REL-006	The system SHALL degrade gracefully when the database is read-only: read endpoints continue to serve; write endpoints return HTTP 503 with retry-after.	—	Should
NFR-REL-007	Background or one-off failures SHALL NOT leak as 500s to the user; the API SHALL return a structured error response with a code (NFR-SEC-007).	—	Must
NFR-REL-008	The system SHALL be idempotent for safe HTTP methods (GET); POST /api/orders SHALL be idempotent	—	Should

	under an idempotency key (FR-ORD-014).		
NFR-REL-009	API instances SHALL be stateless to allow horizontal scaling and zero-downtime rolling deploy.	—	Must
NFR-REL-010	The deployment pipeline SHALL support rollback to the previous version within 5 minutes.	≤ 5 min	Must

3. Verification approach

- Synthetic monitoring of `/health/ready` and a top-level user journey from at least 2 regions.
- Quarterly backup restore drill, documented.
- Chaos drill: kill an API instance mid-traffic during staging load test; assert no requests fail.
- Track MTTR via incident retrospectives.

4. Open questions

- What is the tolerable RTO (Recovery Time Objective) for a full DB rebuild?
- Do we run multi-AZ from day one, or accept single-AZ for MVP?

5. Requirement ↔ Jira ticket mapping

Requirement	Jira Epic	Status
NFR-REL-001 – 010	ECOM-NFR-REL	BACKLOG

6. Related pages

- 3. System Overview & Architecture — deployment topology
- 5.6 Observability & Compliance — monitoring and alerts
- 4.4 Checkout & Order Management — transactional order creation

5.4 Usability & Accessibility

5.4 Usability & Accessibility

Field	Value
Category code	USE
ISO 25010 attribute	Usability
Status	DRAFT

1. Overview

Usability and accessibility NFRs ensure the application is intuitive, responsive across devices, and usable by people relying on assistive technologies.

2. Non-functional requirements

ID	Requirement	Target	Priority
NFR-USE-001	The application SHALL meet WCAG 2.1 Level AA on all primary user flows.	WCAG 2.1 AA	Must
NFR-USE-002	The application SHALL be fully operable using keyboard only on all primary flows.	—	Must
NFR-USE-003	The application SHALL provide visible focus indicators on all interactive elements.	—	Must
NFR-USE-004	All images SHALL have meaningful alt text; decorative	—	Must

	images SHALL be marked as such.		
NFR-USE-005	Form fields SHALL have visible labels and descriptive error messages.	—	Must
NFR-USE-006	Color contrast for text and interactive elements SHALL meet WCAG AA ratios ($\geq 4.5:1$ normal text, $\geq 3:1$ large text and UI components).	—	Must
NFR-USE-007	The UI SHALL be responsive across three breakpoints: mobile (< 768 px), tablet ($768\text{--}1199$ px), desktop (≥ 1200 px).	—	Must
NFR-USE-008	The application SHALL function on the latest 2 major versions of Chrome, Firefox, Safari, and Edge.	—	Must
NFR-USE-009	Long-running actions (> 500 ms) SHALL show a loading indicator; failed actions SHALL show a clear error and a path to retry.	—	Must
NFR-USE-010	Page titles SHALL change per route to support screen-reader	—	Should

	announcement and browser-tab orientation.		
NFR-USE-011	The Lighthouse Accessibility score SHALL be ≥ 90 on all primary pages.	≥ 90	Must
NFR-USE-012	Currency, dates, and numbers SHOULD be locale-aware (Intl APIs); MVP locale is en-US .	—	Should

3. Verification approach

- Lighthouse CI accessibility scan on every PR for the primary pages.
- Manual screen-reader pass (VoiceOver on macOS, NVDA on Windows) per release.
- Keyboard-only walkthrough of all primary flows per release.
- Visual regression check at all three breakpoints (Chromatic or Playwright snapshot).
- Color contrast verified with axe-core in CI.

4. Open questions

- Do we need a high-contrast theme?
- Do we offer reduced-motion preference handling (CSS `prefers-reduced-motion`)?
- Internationalisation scope and rollout timing.

5. Requirement ↔ Jira ticket mapping

Requirement	Jira Epic	Status
NFR-USE-001 – 012	ECOM-NFR-USE	BACKLOG

6. Related pages

- 4.2 Product Catalog — primary visual surface
- 5.1 Performance & Scalability — Lighthouse Perf score
- 7. Acceptance Criteria & Test Strategy — a11y tooling

5.5 Maintainability & Portability

5.5 Maintainability & Portability

Field	Value
Category code	MNT
ISO 25010 attribute	Maintainability, Portability
Status	DRAFT

1. Overview

Maintainability NFRs constrain how the system is structured, tested, documented, and deployed so it can evolve at low risk and cost.

2. Non-functional requirements

ID	Requirement	Target	Priority
NFR-MNT-001	Backend unit + integration test line coverage SHALL be $\geq 80\%$ on changed lines; coverage MUST NOT regress overall.	$\geq 80\%$	Must
NFR-MNT-002	Frontend unit + component test coverage SHALL be $\geq 70\%$.	$\geq 70\%$	Must
NFR-MNT-003	The backend SHALL be lint-clean (ruff or flake8) and type-clean (mypy strict on services).	—	Must
NFR-MNT-004	The frontend SHALL be lint-clean (ESLint)	—	Must

	and type-clean if TypeScript is adopted; JSX components SHALL be ESLint-clean if plain JS.		
NFR-MNT-005	The backend SHALL adopt Alembic database migrations from v1.1; schema changes SHALL ship as migration scripts.	—	Must (v1.1)
NFR-MNT-006	The application SHALL be runnable end-to-end via a single bootstrap command (e.g., <code>make dev</code> or <code>docker compose up</code>).	—	Must
NFR-MNT-007	The backend SHALL be deployable as a Docker image with a documented Dockerfile.	—	Must
NFR-MNT-008	The frontend SHALL be deployable as static assets to any static host or CDN.	—	Must
NFR-MNT-009	The system SHALL be portable from SQLite (dev) to PostgreSQL (prod) without code changes — connection string only.	—	Must

NFR-MNT-010	Configuration SHALL be 12-factor: environment variables only; no per-environment code branches.	—	Must
NFR-MNT-011	Public APIs (Pydantic schemas) SHALL be documented in OpenAPI; breaking changes SHALL be versioned via path prefix.	—	Must
NFR-MNT-012	Source code SHALL be organised by feature (routers, services, schemas per area) rather than by layer-only.	—	Should
NFR-MNT-013	The repository SHALL include a README, SETUP_GUIDE, and ARCHITECTURE document; these SHALL be reviewed each release.	—	Must
NFR-MNT-014	Cyclomatic complexity SHOULD be ≤ 10 per function; functions exceeding this MUST be refactored or annotated with rationale.	≤ 10	Should

3. Verification approach

- Coverage gate in CI (pytest-cov, Vitest coverage).
- Lint and type checks in CI (ruff, mypy, ESLint).
- Docker image build runs in CI on every PR.
- Onboarding rehearsal: a new contributor runs the bootstrap and is productive within 1 hour.

4. Open questions

- Is TypeScript adoption in scope for v1.1, or deferred?
- Do we adopt pre-commit hooks repo-wide, or rely on CI only?

5. Requirement ↔ Jira ticket mapping

Requirement	Jira Epic	Status
NFR-MNT-001 – 014	ECOM-NFR-MNT	BACKLOG

6. Related pages

- 3. System Overview & Architecture — code organisation
- 7. Acceptance Criteria & Test Strategy — test tooling

5.6 Observability & Compliance

5.6 Observability & Compliance

Field	Value
Category code	OBS
ISO 25010 attribute	(cross-cutting)
Status	DRAFT

1. Overview

Observability NFRs make the running system's behaviour visible: logs, metrics, traces, and audit events. Compliance NFRs codify what we must keep, expose, or destroy on user request.

2. Non-functional requirements

ID	Requirement	Target	Priority
NFR-OBS-001	Every API request SHALL produce a structured (JSON) log entry with: timestamp, request id, method, path, status, latency, user id (if authenticated).	100% of requests	Must
NFR-OBS-002	Logs SHALL include a request-correlation id that propagates to downstream calls (and to the frontend in dev).	—	Must
NFR-OBS-003	Application metrics SHALL be exposed in Prometheus format at <code>/metrics</code> : request count, error rate,	—	Should

	latency histogram per endpoint.		
NFR-OBS-004	The system SHOULD emit distributed traces (OpenTelemetry) for at least the order-creation transaction.	—	Should
NFR-OBS-005	Security-relevant events (login success / failure, admin actions, role changes) SHALL be written to an audit log distinct from request logs.	—	Must
NFR-OBS-006	Logs SHALL be retained for ≥ 30 days; audit logs SHALL be retained for ≥ 1 year.	30 d / 365 d	Must
NFR-OBS-007	Alerts SHALL fire on: error rate $> 1\%$ over 5 min, p95 latency exceeding NFR-PERF-001 by 50% for 10 min, readiness probe failing for 2 min.	—	Must
NFR-OBS-008	The system SHALL surface uncaught exceptions to an error-tracking service (Sentry-equivalent) without exposing them to users.	—	Should
NFR-OBS-009	The system SHALL provide a self-service	—	Should

	endpoint or runbook for GDPR data-subject access and erasure requests (see NFR-SEC-012).		
NFR-OBS-010	The system SHALL document its data retention and deletion policy in a published privacy note.	—	Should
NFR-OBS-011	The system SHALL avoid logging PII at INFO / DEBUG level; PII MUST be redacted or hashed.	—	Must
NFR-OBS-012	Dashboards SHALL exist for: request rate, error rate, p95 latency, active user count, order rate.	—	Should

3. Verification approach

- Log sampling review per release: confirm structured format, presence of request id, absence of PII.
- Alert fire-drill quarterly (simulate latency, simulate readiness failure).
- GDPR runbook walkthrough annually.

4. Open questions

- Which logging backend (Loki, ELK, hosted)?
- Which metrics / tracing backend (self-hosted Prometheus + Grafana vs hosted)?
- Is there a regulator-driven retention floor we must respect (PCI, regional eCommerce law)?

5. Requirement ↔ Jira ticket mapping

Requirement	Jira Epic	Status
-------------	-----------	--------

6. Related pages

- 5.2 Security & Privacy — audit-related security NFRs
- 5.3 Reliability & Availability — alerts and SLOs
- 3. System Overview & Architecture — operational topology

6. External Interfaces & Integrations

6. External Interfaces & Integrations

Field	Value
Section	6 of 8
Status	DRAFT

6.1 Overview

This section catalogs all interfaces the application exposes or consumes. In v1 only the REST API is exposed; planned third-party integrations are documented here so they can be designed in without rework.

6.2 REST API conventions

Convention	Rule
Base path	<code>/api</code>
Versioning	Path-based when introduced (<code>/api/v2/...</code>); v1 is implicit
Content type	<code>application/json</code> for request and response bodies
Authentication	Bearer JWT in <code>Authorization</code> header
Error envelope	<code>{ "detail": "...", "code": "...", "context": {...} }</code>
Pagination	Cursor- or page-based query params (planned for catalog scale)
Idempotency	Idempotency-Key header on order creation (planned)
Documentation	OpenAPI 3.x via FastAPI at <code>/docs</code> and <code>/redoc</code>

6.3 HTTP status code policy

Twilio	SMS order-status alerts (optional)	Considered	Cost / opt-in concerns
S3-compatible object store	Product image hosting	Planned	Currently URLs only
OAuth providers (Google, GitHub)	Social login	Future	Pairs with FR-AUTH future scope

6.6 Stripe integration sketch (planned)

Error loading the extension!

Mermaid

```

1 sequenceDiagram
2     actor User
3     participant SPA as React SPA
4     participant API as FastAPI
5     participant Stripe
6
7     User->>SPA: Click "Pay"
8     SPA->>API: POST /api/orders (intent=create)
9     API->>Stripe: PaymentIntent.create(amount)
10    Stripe-->>API: client_secret
11    API-->>SPA: order_id, client_secret
12    SPA->>Stripe: confirmCardPayment(client_secret)
13    Stripe-->>SPA: success | requires_action
14    SPA->>API: POST /api/orders/{id}/confirm
15    API->>Stripe: retrieve(intent) verify status
16    Stripe-->>API: succeeded
17    API-->>SPA: order confirmed
18    Note over API,Stripe: Webhook also confirms async

```

6.7 Webhook interfaces (future)

- **Stripe → API:** `payment_intent.succeeded` / `.payment_failed` events update order status.
- **API → operator endpoint:** order events for downstream fulfilment (out of scope for MVP).

7. Acceptance Criteria & Test Strategy

7. Acceptance Criteria & Test Strategy

Field	Value
Section	7 of 8
Owner	QA Engineering
Status	DRAFT

7.1 Test philosophy

- Every requirement (FR or NFR) has at least one verifying test or check.
- Tests are written against requirement IDs, not against implementation details.
- Automation is the default; manual testing is reserved for exploratory and accessibility verification.
- CI is the gate: no merge to main without green tests.

7.2 Test pyramid

Error loading the extension!

✓ Mermaid

```
1 flowchart TB
2   E2E[E2E tests ~5 percent<br>Playwright]
3   INT[Integration / API tests ~25 percent<br>pytest + httpx]
4   UNIT[Unit tests ~70 percent<br>pytest backend, Vitest frontend]
5   UNIT --> INT --> E2E
6   style E2E fill:#fde,stroke:#a45
7   style INT fill:#fed,stroke:#a85
8   style UNIT fill:#efd,stroke:#5a4
```

7.3 Test users (seeded)

Username	Password	Profile	Use in tests
standard_user	secret_sauce	Normal happy-path shopper	Default user for all golden-path tests
problem_user	secret_sauce	Triggers selected UI quirks	Negative / visual-regression tests

performance_user	secret_sauce	Used for load / perf scenarios	NFR-PERF verification
------------------	--------------	--------------------------------	-----------------------

7.4 Golden-path scenarios

ID	Scenario	Covers
GP-01	Register → login → browse → add 2 items → checkout → view in order history	FR-AUTH-001/002/004, FR-CAT-001, FR-CART-001/002/004, FR-ORD-001/008
GP-02	Login existing user → cart persists from prior session → adjust qty → checkout	FR-AUTH-002, FR-CART-003/005, FR-ORD-001
GP-03	Filter catalog by category → search by keyword → open product detail	FR-CAT-002/003/005
GP-04	Add out-of-stock product → expect 409	FR-CART-009
GP-05	Token expires mid-session → user is redirected to login	FR-AUTH-006, FR-CART-012

7.5 NFR verification approach

NFR area	Method	Tooling
Performance	Load test against staging at expected and peak load profiles	k6 or Locust
Security	SAST in CI; periodic manual review; OWASP ZAP scan in staging	Bandit, npm audit, OWASP ZAP
Reliability	Chaos drill (kill API mid-request); restore-from-backup drill	Manual scripted

Usability / a11y	Lighthouse CI; manual screen-reader pass; keyboard-only pass	Lighthouse, axe-core, VoiceOver / NVDA
Maintainability	Coverage gate in CI; lint and type checks	pytest-cov, Vitest, ruff, mypy, eslint
Observability	Sample log review; alert-rule fire drill	Manual + dashboard review

7.6 CI / CD integration

- **Build** — backend lints, types, tests; frontend lints, builds, tests.
- **Smoke** — golden-path GP-01 run against an ephemeral environment on every PR.
- **Coverage gate** — fail under thresholds in NFR-MNT.
- **SAST gate** — fail on any critical security finding.
- **Deploy** — staging on main merge; production on tagged release.

7.7 Definition of Done (per story)

- ☐ Code merged behind passing CI
- ☐ Unit tests written and passing
- ☐ Integration test exists if the story touches an API endpoint
- ☐ All acceptance criteria on the requirement page checked off
- ☐ No new SAST critical findings
- ☐ Requirement page's Jira mapping table updated

8. Roadmap & Out-of-Scope (v2)

8. Roadmap & Out-of-Scope (v2)

Field	Value
Section	8 of 8
Status	DRAFT

8.1 MVP recap

MVP delivers a full end-to-end shopper experience without payment capture and without an admin surface: auth, catalog, cart, checkout (records orders), order history. Detailed scope in Sections 1.2 and 2.7.

8.2 v1.1 (post-MVP hardening)

- Alembic-based database migrations.
- Redis cache for the catalog GET path.
- Pagination on product list endpoint.
- Refresh-token endpoint to lengthen sessions without long-lived JWTs.
- Email-on-order-placed (SendGrid or local SMTP).
- Rate-limiting on auth endpoints (NFR-SEC-009).

8.3 v2 (capability expansion)

1. **Payments** — Stripe integration replacing the placeholder `payment_method` string.
2. **Admin surface** — see 4.5 *Admin / Inventory*; CRUD for products, orders view, inventory adjustments.
3. **Reviews and ratings** — 1–5 stars and text on product detail pages.
4. **Wishlist / favourites** — saved products per user.
5. **OAuth login** — Google, GitHub social providers.
6. **Coupon / discount codes** — applied at checkout.
7. **Recommendations** — "Customers also bought..." panel.
8. **Order tracking** — fulfilment status updates with email notifications.

8.4 Explicitly out of scope

Item	Why excluded
Native mobile apps	Responsive web covers all primary devices
B2B (multi-buyer / approval workflows)	Off-mission
Multi-currency / multi-language (i18n)	Adds complexity; defer to clear demand
Marketplace (third-party sellers)	Out of model
Subscription / recurring billing	Off-mission for a transactional store
Custom checkout fields (gift wrap, gift notes)	Defer to clear demand
Offline mode / PWA	Not required for the target audience

8.5 Indicative timeline

Error loading the extension!

Mermaid

```

1 gantt
2   title SauceDemo eCommerce indicative roadmap
3   dateFormat YYYY-MM-DD
4   section MVP
5     Auth and Catalog      :done,    a1, 2026-01-01, 30d
6     Cart and Checkout     :done,    a2, after a1, 21d
7     NFR hardening        :active,   a3, after a2, 14d
8   section v1.1
9     Migrations and cache  :         b1, after a3, 14d
10    Email notifications   :         b2, after b1, 10d
11    Rate limiting         :         b3, after b1, 7d
12  section v2
13    Stripe payments       :         c1, after b2, 21d
14    Admin surface         :         c2, after b2, 30d
15    Reviews and wishlist  :         c3, after c1, 21d

```

 The timeline above is illustrative for planning conversations. Replace with the actual sprint cadence once committed.

Requirements ↔ Jira Ticket Mapping

Requirements ↔ Jira Ticket Mapping

 This page maps every wiki requirement (FR-*, NFR-*) to its corresponding Jira issue in the **ECOM** project. Use this as the single source of truth when cross-referencing between docs and tickets.

Summary

Metric	Count
Total Jira issues created	154
Epics	11 (ECOM-6 → ECOM-16)
Functional stories (FR-*)	70 (ECOM-17 → ECOM-86)
Non-functional tasks (NFR-*)	73 (ECOM-87 → ECOM-159)
Project	ECOM (AgenticQAJIRA)
Status	CREATED

Epic index

Epic key	Name	ID prefix	Children	Priority
ECOM-6	[AUTH] Authentication & User Management	FR-AUTH-*	15 stories	MVP
ECOM-7	[CAT] Product Catalog & Browsing	FR-CAT-*	13 stories	MVP
ECOM-8	[CART] Shopping Cart	FR-CART-*	15 stories	MVP
ECOM-9	[ORD] Checkout & Order	FR-ORD-*	15 stories	MVP

	Management			
ECOM-10	[ADM] Admin & Inventory	FR-ADM-*	12 stories	v2
ECOM-11	[NFR-PERF] Performance & Scalability	NFR-PERF-*	10 tasks	MVP
ECOM-12	[NFR-SEC] Security & Privacy	NFR-SEC-*	15 tasks	MVP
ECOM-13	[NFR-REL] Reliability & Availability	NFR-REL-*	10 tasks	MVP
ECOM-14	[NFR-USE] Usability & Accessibility	NFR-USE-*	12 tasks	MVP
ECOM-15	[NFR-MNT] Maintainability & Portability	NFR-MNT-*	14 tasks	MVP
ECOM-16	[NFR-OBS] Observability & Compliance	NFR-OBS-*	12 tasks	MVP

Functional requirements — Stories

FR-AUTH-* (Epic ECOM-6 — Authentication & User Management)

Requirement	Jira	Summary	Priority
FR-AUTH-001	ECOM-17	Register new user account	Must
FR-AUTH-002	ECOM-18	Enforce uniqueness of username and email	Must
FR-AUTH-003	ECOM-19	Bcrypt password hashing	Must

FR-AUTH-004	ECOM-20	JWT login for registered users	Must
FR-AUTH-005	ECOM-21	Generic 401 on invalid credentials	Must
FR-AUTH-006	ECOM-22	JWT signed HS256 with 30-min expiry	Must
FR-AUTH-007	ECOM-23	GET /api/auth/me returns profile	Must
FR-AUTH-008	ECOM-24	Reject protected endpoints without valid JWT	Must
FR-AUTH-009	ECOM-25	Frontend stores JWT in localStorage	Must
FR-AUTH-010	ECOM-26	Auto-redirect to login on 401	Must
FR-AUTH-011	ECOM-27	Password complexity: 8+ chars, letter + digit	Must
FR-AUTH-012	ECOM-28	Seed three test users for automated testing	Must
FR-AUTH-013	ECOM-29	Refresh-token mechanism	Should (v1.1)
FR-AUTH-014	ECOM-30	Password-reset via email link	Should (v1.1)
FR-AUTH-015	ECOM-31	OAuth 2.0 social login	Could (v2)

FR-CAT-* (Epic ECOM-7 — Product Catalog & Browsing)

Requirement	Jira	Summary	Priority
FR-CAT-001	ECOM-32	List all active products	Must
FR-CAT-002	ECOM-33	Keyword search on product name and	Must

		description	
FR-CAT-003	ECOM-34	Filter products by category	Must
FR-CAT-004	ECOM-35	Combine search and category filters	Must
FR-CAT-005	ECOM-36	Retrieve single product detail	Must
FR-CAT-006	ECOM-37	Surface inventory / in-stock state	Must
FR-CAT-007	ECOM-38	Exclude inactive products	Must
FR-CAT-008	ECOM-39	Public read, admin-only mutations	Must
FR-CAT-009	ECOM-40	Seed at least 6 demo products in ≥ 2 categories	Must
FR-CAT-010	ECOM-41	Paginate product list at > 50 items	Should (v1.1)
FR-CAT-011	ECOM-42	Deterministic default sort + sort query param	Should
FR-CAT-012	ECOM-43	Frontend placeholder for broken product images	Should
FR-CAT-013	ECOM-44	Category facets aggregation	Could

FR-CART-* (Epic ECOM-8 — Shopping Cart)

Requirement	Jira	Summary	Priority
FR-CART-001	ECOM-45	Add product to authenticated user's	Must

		cart	
FR-CART-002	ECOM-46	Increment qty when adding existing product	Must
FR-CART-003	ECOM-47	Persist cart server-side per user	Must
FR-CART-004	ECOM-48	Retrieve cart with totals and item details	Must
FR-CART-005	ECOM-49	Update line quantity	Must
FR-CART-006	ECOM-50	Reject quantity ≤ 0 with HTTP 400	Must
FR-CART-007	ECOM-51	Remove single cart line	Must
FR-CART-008	ECOM-52	Clear entire cart in one operation	Must
FR-CART-009	ECOM-53	Block adding inactive or out-of-stock product	Must
FR-CART-010	ECOM-54	Live cart item-count badge in header	Must
FR-CART-011	ECOM-55	Auto-clear cart after successful order creation	Must
FR-CART-012	ECOM-56	Reject cart endpoints without valid JWT	Must
FR-CART-013	ECOM-57	Cap line quantity at current inventory	Should
FR-CART-014	ECOM-58	Confirmation prompt before clearing cart	Should

FR-CART-015	ECOM-59	Flag unavailable lines and exclude from total	Could
-------------	-------------------------	---	-------

FR-ORD-* (Epic ECOM-9 — Checkout & Order Management)

Requirement	Jira	Summary	Priority
FR-ORD-001	ECOM-60	Create order from current cart	Must
FR-ORD-002	ECOM-61	Reject order creation when cart is empty	Must
FR-ORD-003	ECOM-62	Validate stock and active state at order creation	Must
FR-ORD-004	ECOM-63	Atomically decrement inventory at order creation	Must
FR-ORD-005	ECOM-64	Clear cart on successful order creation	Must
FR-ORD-006	ECOM-65	Snapshot unit prices into order lines	Must
FR-ORD-007	ECOM-66	Compute and store order total_amount	Must
FR-ORD-008	ECOM-67	List authenticated user's orders	Must
FR-ORD-009	ECOM-68	Order detail endpoint with line items	Must
FR-ORD-010	ECOM-69	Deny foreign-user orders with 404	Must
FR-ORD-011	ECOM-70	Order status lifecycle	Must

FR-ORD-012	ECOM-71	Persist shipping address per order	Must
FR-ORD-013	ECOM-72	Cancel PENDING/CONFIRMED orders and restore stock	Should
FR-ORD-014	ECOM-73	Idempotency key on order creation	Should
FR-ORD-015	ECOM-74	Email confirmation on order creation	Could

FR-ADM-* (Epic ECOM-10 — Admin & Inventory, v2)

Requirement	Jira	Summary	Priority
FR-ADM-001	ECOM-75	Add role attribute to User	Must (v2)
FR-ADM-002	ECOM-76	Admin can create products	Must (v2)
FR-ADM-003	ECOM-77	Admin can update product fields	Must (v2)
FR-ADM-004	ECOM-78	Soft-delete products only	Must (v2)
FR-ADM-005	ECOM-79	Admin can adjust inventory with audit	Must (v2)
FR-ADM-006	ECOM-80	Admin lists all orders across users	Must (v2)
FR-ADM-007	ECOM-81	Admin transitions order status	Must (v2)
FR-ADM-008	ECOM-82	Non-admin endpoints reject with 403	Must (v2)

FR-ADM-009	ECOM-83	Admin can deactivate user accounts	Should (v2)
FR-ADM-010	ECOM-84	Admin metrics endpoint	Should (v2)
FR-ADM-011	ECOM-85	CSV export of orders	Could (v2)
FR-ADM-012	ECOM-86	Audit log for all admin actions	Must (v2)

Non-functional requirements — Tasks

NFR-PERF-* (Epic ECOM-11 — Performance & Scalability)

Requirement	Jira	Summary
NFR-PERF-001	ECOM-87	API latency targets (p50/p95/p99)
NFR-PERF-002	ECOM-88	Catalog reads sustain ≥ 100 RPS
NFR-PERF-003	ECOM-89	Checkout p95 ≤ 500 ms
NFR-PERF-004	ECOM-90	Lighthouse Performance ≥ 85 on mobile
NFR-PERF-005	ECOM-91	Frontend bundle ≤ 200 KB gzipped
NFR-PERF-006	ECOM-92	First Contentful Paint ≤ 1.5 s on fast-3G
NFR-PERF-007	ECOM-93	Cache-Control on catalog list
NFR-PERF-008	ECOM-94	Horizontal scaling: ≥ 4 stateless API instances
NFR-PERF-009	ECOM-95	Redis cache for catalog reads (v1.1)
NFR-PERF-010	ECOM-96	Catalog list pagination at > 50 products

NFR-SEC-* (Epic ECOM-12 — Security & Privacy)

Requirement	Jira	Summary
NFR-SEC-001	ECOM-97	HTTPS / TLS 1.2+ in non-local environments
NFR-SEC-002	ECOM-98	Bcrypt password hashing ≥ 12 rounds
NFR-SEC-003	ECOM-99	JWT secret ≥ 256 -bit entropy, env-only
NFR-SEC-004	ECOM-100	JWT access ≤ 30 min, refresh ≤ 7 days
NFR-SEC-005	ECOM-101	Pydantic validation + ORM-only SQL
NFR-SEC-006	ECOM-102	CORS explicit allow-list per env
NFR-SEC-007	ECOM-103	No stack traces in prod error responses
NFR-SEC-008	ECOM-104	Rotate secrets every 90 days
NFR-SEC-009	ECOM-105	Rate limiting on auth endpoints
NFR-SEC-010	ECOM-106	Security headers on API + frontend
NFR-SEC-011	ECOM-107	PII exposed only via owner-scoped endpoints
NFR-SEC-012	ECOM-108	GDPR data-deletion runbook
NFR-SEC-013	ECOM-109	No PII / secrets in logs
NFR-SEC-014	ECOM-110	CVE scanning in CI
NFR-SEC-015	ECOM-111	SAST in CI

NFR-REL-* (Epic ECOM-13 — Reliability & Availability)

Requirement	Jira	Summary
-------------	------	---------

NFR-REL-001	ECOM-112	≥ 99.5% monthly uptime SLO
NFR-REL-002	ECOM-113	Liveness + readiness probes
NFR-REL-003	ECOM-114	Transactional order creation
NFR-REL-004	ECOM-115	Daily DB backups + quarterly restore drill
NFR-REL-005	ECOM-116	MTTR ≤ 30 minutes
NFR-REL-006	ECOM-117	Read-only DB graceful degradation
NFR-REL-007	ECOM-118	Structured error response (no 500 leaks)
NFR-REL-008	ECOM-119	Idempotency on safe methods + POST orders
NFR-REL-009	ECOM-120	Stateless API for horizontal scaling
NFR-REL-010	ECOM-121	Rollback within 5 minutes

NFR-USE-* (Epic ECOM-14 — Usability & Accessibility)

Requirement	Jira	Summary
NFR-USE-001	ECOM-122	WCAG 2.1 Level AA on primary flows
NFR-USE-002	ECOM-123	Keyboard-only operation on primary flows
NFR-USE-003	ECOM-124	Visible focus indicators
NFR-USE-004	ECOM-125	Meaningful alt text on all images
NFR-USE-005	ECOM-126	Visible form labels + descriptive errors
NFR-USE-006	ECOM-127	Color contrast meets WCAG AA ratios

NFR-USE-007	ECOM-128	Responsive across mobile / tablet / desktop
NFR-USE-008	ECOM-129	Browser support: latest 2 versions
NFR-USE-009	ECOM-130	Loading indicators and retry paths
NFR-USE-010	ECOM-131	Per-route page titles
NFR-USE-011	ECOM-132	Lighthouse Accessibility ≥ 90
NFR-USE-012	ECOM-133	Locale-aware currency / dates / numbers

NFR-MNT-* (Epic ECOM-15 — Maintainability & Portability)

Requirement	Jira	Summary
NFR-MNT-001	ECOM-134	Backend test coverage $\geq 80\%$
NFR-MNT-002	ECOM-135	Frontend test coverage $\geq 70\%$
NFR-MNT-003	ECOM-136	Backend lint-clean + type-clean
NFR-MNT-004	ECOM-137	Frontend lint-clean (ESLint)
NFR-MNT-005	ECOM-138	Alembic migrations
NFR-MNT-006	ECOM-139	Single-command bootstrap
NFR-MNT-007	ECOM-140	Backend Dockerfile + container image
NFR-MNT-008	ECOM-141	Frontend deployable as static assets
NFR-MNT-009	ECOM-142	SQLite $\rightarrow$ Postgres portability
NFR-MNT-010	ECOM-143	12-factor config
NFR-MNT-011	ECOM-144	OpenAPI docs + path versioning

NFR-MNT-012	ECOM-145	Feature-based code organisation
NFR-MNT-013	ECOM-146	README / SETUP / ARCHITECTURE kept current
NFR-MNT-014	ECOM-147	Cyclomatic complexity $\leq 10/\text{fn}$

NFR-OBS-* (Epic ECOM-16 — Observability & Compliance)

Requirement	Jira	Summary
NFR-OBS-001	ECOM-148	Structured JSON request logs
NFR-OBS-002	ECOM-149	Request-correlation id propagation
NFR-OBS-003	ECOM-150	Prometheus /metrics endpoint
NFR-OBS-004	ECOM-151	OpenTelemetry traces for order creation
NFR-OBS-005	ECOM-152	Separate audit log for security events
NFR-OBS-006	ECOM-153	Log retention: 30d / 365d audit
NFR-OBS-007	ECOM-154	Alerts on error rate / p95 / readiness
NFR-OBS-008	ECOM-155	Error tracking service (Sentry-equivalent)
NFR-OBS-009	ECOM-156	GDPR data-subject runbook
NFR-OBS-010	ECOM-157	Published data retention & deletion policy
NFR-OBS-011	ECOM-158	No PII at INFO/DEBUG log level
NFR-OBS-012	ECOM-159	Operational dashboards

Useful JQL queries

Query	JQL
All MVP work	<code>project = ECOM AND labels = mvp</code>
All v2 work	<code>project = ECOM AND labels = v2</code>
All v1.1 work	<code>project = ECOM AND labels = v1.1</code>
All security-related	<code>project = ECOM AND labels in (security, "nfr-sec")</code>
All frontend	<code>project = ECOM AND labels = frontend</code>
By area (e.g. Cart)	<code>project = ECOM AND labels = "fr-cart"</code>
All Must-priority	<code>project = ECOM AND labels = must</code>
Open epics	<code>project = ECOM AND issuetype = Epic</code>

Note on individual page mapping tables

- 📄 The "Requirement ↔ Jira ticket mapping" placeholder tables on the individual requirement pages (4.1, 4.2, ...) still show `TBD` and `BACKLOG` as the original templates. This master page supersedes those — the per-page tables can be updated lazily or left as-is, since this page is the canonical mapping.